Flink SQL reports

Seven deterministic reports (plus an optional eighth, AI-generated one — see Optional: AI root-cause analysis) that read two streams of isotope metadata — the **produce side** (the `x-isotope-*` headers stamped on every event topic record by `IsotopeProducerInterceptor`) and the **consume side** (the value-less marker records on `isotope_consume_edge_markers` emitted by `IsotopeContext.recordConsume`) — and surface what's flowing where, how fast, and how reliably. Each report runs as a long-lived `INSERT INTO <report>_1m SELECT …` streaming job. The aggregation logic is identical across runtimes; only the source/sink DDL and function-registration glue differ.

The headline addition is the `bipartite_topology` report: it unions the produce and consume views to render the pipeline as a literal **bipartite graph** from graph theory — services in one vertex set, topics in the other, edges crossing between them in both directions. See root README §1.0 "How an Isotope Traverses an Event Pipeline" for the full motivation.

Runtime	Reports	Sink format	Where the SQL lives
Confluent Platform Flink (Flink 2.1 CMF Application on Minikube)	7 (latency, topology, bipartite-topology, hop-distribution, coverage, stuck-trace, latency-percentiles)	`avro-confluent` (SR-framed Avro)	scripts/flink/sql/cp/ — run by `IsotopeReportsJob`, deployed by scripts/deploy-cmf-flink-reports.sh
Confluent Cloud for Apache Flink (CCAF)	7 (same set as CP)	`proto-registry` (SR-framed Protobuf)	Inlined as `confluent_flink_statement` resources in terraform/setup-confluent-flink.tf — applied by scripts/deploy-cc-flink-reports.sh

Control Center deserializes both sink formats natively.

Table of Contents

1.0 What each report computes

Every report aggregates over a `TUMBLE(event_time, INTERVAL '1' MINUTE)` window. The "Source view" column names the typed view (or views) the report reads from; the views in turn filter the single `isotope_raw` source table by the presence/absence of the `x-isotope-consumer-service` header.

Every report also carries a **`pipeline`** column (decoded from the `x-isotope-pipeline` header) as a leading grouping dimension, so rows for an `orders` pipeline and a `location` pipeline never aggregate together. The "What it computes" column below lists the *additional* keys each report groups by, on top of `pipeline`.

Report	Source view	What it computes	Runtimes
`latency`	`isotope`	avg / min / max end-to-end latency by `origin_service` × `current_topic`	CP, CCAF
`topology`	`isotope`	produce-edge counts per `(producer_service, topic)` per minute	CP, CCAF
`bipartite_topology`	`isotope` and `consume_events`	full bipartite graph: produce edges plus consume edges per minute	CP, CCAF
`hop_distribution`	`isotope`	record counts bucketed by `hop_count` per topic per minute	CP, CCAF
`coverage`	`isotope`	distinct traces per topic per minute	CP, CCAF
`stuck_trace`	`isotope`	alerts (via `STUCK_TRACE_PTF`) for traces idle ≥60s of event time	CP, CCAF
`latency_percentiles`	`isotope`	p50 / p95 / p99 (via `LATENCY_PERCENTILES` PTF, T-Digest)	CP, CCAF

2.0 Optional: AI root-cause analysis (CCAF-only, off by default)

Beyond the seven deterministic reports, an **eighth, AI-generated report** turns each stuck-trace *alert* into a natural-language root-cause hypothesis plus a one-line remediation. It's a CCAF-only feature, wired by terraform/setup-ccaf-ai.tf and **gated on `var.enable_trace_rca` (default `false`)**, so a normal deploy is unaffected.

When enabled, it adds three `confluent_flink_statement` resources:

- `CREATE MODEL trace_rca` — registers a remote text-generation model.
- A `proto-registry` sink table `isotope_report_trace_rca_1m`.
- An `INSERT … SELECT … LATERAL TABLE(ML_PREDICT('trace_rca', …))` that calls the model **once per alert** (reading the low-volume 1-minute stuck-trace report topic, never the source stream) and writes the hypothesis to its own topic — it never overwrites the deterministic reports.

The default provider is OpenAI (`gpt-4o`). Claude is supported two ways: directly via **Anthropic** (`rca_model_provider = "anthropic"`, endpoint `https://api.anthropic.com/v1/messages`, a bare `rca_model_api_key`, plus the required `rca_model_max_tokens`), or via **AWS Bedrock** (`rca_model_provider = "bedrock"` with AWS credentials). Other supported providers: `vertexai`, `azureopenai`, `googleai`, `sagemaker`, `azureml`.

The standalone SQL walkthrough — a `CREATE MODEL` + `ML_PREDICT` PoC with provider notes and alternative options — is in sql/ccaf-ai/trace_rca.fql. See also root README §4.5.

3.0 Format-by-runtime, not by domain

The sink **format** differs by runtime for one platform-level reason:

- **cp-flink ships SR-Avro but not SR-Protobuf.** Apache Flink open-source publishes `flink-sql-avro-confluent-registry`; there is no SR-integrated Protobuf equivalent. We could hand-write one (~150 lines wrapping `flink-protobuf` with magic-byte framing + SR client) but Avro already gives us Control Center decoding with zero custom code. CCAF has SR-Protobuf, so its sinks use it.

The report **set** is identical on both runtimes — but that took a deliberate choice for percentiles: **CCAF rejects all user-defined aggregate functions** with `aggregate functions are not supported`, regardless of accumulator shape. So `LATENCY_PERCENTILES` is implemented as a `ProcessTableFunction` (T-Digest sketch + per-window state and timers), not an aggregate function — a PTF registers and runs on both runtimes, same as `STUCK_TRACE_PTF`.

The demo *event* topics (`orders.placed`, `orders.enriched`, `orders.fulfilled`) ride Protobuf+SR via the Java app's `DemoEvent` schema on both runtimes — those are written by the Kafka producer client, not by Flink, so the SR-Protobuf gap doesn't apply.

4.0 Layout

```
scripts/flink/sql/cp/                    CP Flink — session-cluster SQL
  00_source_table.fql                    CREATE TABLE per topic ('connector'
= 'kafka') + isotope_raw UNION view
  01_register_functions.fql              CREATE FUNCTION … USING JAR
'file:///opt/flink/lib/isotope-flink-udf.jar'
  05_isotope_view.fql                    Typed view; decodes x-isotope-*
header scalars (produces only)
  06_consume_events_view.fql             Typed view of
isotope_consume_edge_markers markers (consume edges)
  05_report_sinks.fql                    CREATE TABLE for each
isotope_report_*_1m Kafka sink (avro-confluent)
  10_latency_report.fql                  INSERT INTO: avg/min/max latency by
origin × topic
  20_topology_report.fql                 INSERT INTO: produce-edge counts
per minute
  25_bipartite_topology_report.fql       INSERT INTO: produce plus consume
edges (bipartite graph) per minute
  30_hop_distribution.fql                INSERT INTO: hop-count buckets per
topic per minute
```

```
  40_coverage_report.fql                INSERT INTO: distinct traces per
topic per minute
  60_stuck_trace_report.fql             INSERT INTO: stuck-trace alerts via
STUCK_TRACE_PTF
  70_latency_percentiles_report.fql     INSERT INTO: p50/p95/p99 via
LATENCY_PERCENTILES PTF (T-Digest)
  99_teardown.fql                       DROP TABLE/VIEW/FUNCTION (companion
to flink-reports-down)

scripts/flink/sql/ccaf-ai/            Optional AI report — CCAF only (see
section below)
  trace_rca.fql                         CREATE MODEL + ML_PREDICT PoC: LLM
root-cause hypothesis per stuck-trace alert
```

CCAF runs the same seven reports, but the SQL lives inline as `confluent_flink_statement` resources in terraform/setup-confluent-flink.tf — 25 statements: ALTER (×4) + raw view + typed produce view + typed consume view + 7 sinks + `STUCK_TRACE_PTF` and `LATENCY_PERCENTILES` drop+register (×2 each = 4) + 7 INSERTs. The JAR is uploaded as a `confluent_flink_artifact` and referenced via `USING JAR 'confluent-artifact://<id>'`.

5.0 Wire-format detail (CP only)

Window columns ride as `BIGINT` epoch millis on the wire, not `TIMESTAMP_LTZ`. Flink 2.1.2's `avro-confluent` schema-derivation path raises `UnsupportedOperationException: Unsupported to derive Schema for type: TIMESTAMP_LTZ(3)` for that type. We cast in the INSERT (`UNIX_TIMESTAMP(CAST(window_start AS STRING)) * 1000`) so the on-wire schema is plain Avro `long`. Consumers rehydrate via `TO_TIMESTAMP_LTZ(window_start, 3)`. The CCAF Protobuf sinks don't hit this — `proto-registry` handles `TIMESTAMP_LTZ` directly.

6.0 PTF JAR

The single shadow JAR `ptf/build/libs/isotope-flink-udf.jar` (produced by `./gradlew :ptf:shadowJar`) carries both JAR-backed functions — both `ProcessTableFunction`s — under the `ai.signalroom.kafka.isotope.flink` package:

- `LatencyPercentilesPTF` (registered as `LATENCY_PERCENTILES`) — T-Digest p50/p95/p99 via per-window state + event-time timers.
- `StuckTracePTF` — per-trace state + event-time timer that emits alerts for traces idle ≥60s.

Both register on both runtimes; only the `CREATE FUNCTION … USING JAR …` clause differs (`file://` path on CP, `confluent-artifact://` reference on CCAF).

7.0 Operations

7.1 CP Flink (Minikube)

```
make flink-up           # cert-manager → CFK Flink Operator → MinIO → CMF
2.4 → env → app image
make kafka-pf-up        # localhost:30092 → Kafka, localhost:8081 → SR
```

```
make flink-reports-up   # build app JAR → upload as cmf:// artifact →
deploy the CMF Application
make flink-reports-down # delete the CMF Application + artifact + sink
topics
make flink-down         # tear down the application, CMF, MinIO, operator,
cert-manager
```

The 7 reports run as a single Flink 2.1 CMF **Application** (`isotope-reports`, entry point `IsotopeReportsJob`) — visible in CMF and Control Center's Flink tab.

7.2 CCAF (Confluent Cloud, Terraform-driven)

```
make cc-flink-reports-up   CONFLUENT_API_KEY=... CONFLUENT_API_SECRET=...
                           # terraform apply: env + cluster + topics +
compute pool + artifact + 25 statements
                           # also regenerates terraform/terraform.png via
`terraform graph | dot`
source scripts/cc-cli-env.sh          # exports BOOTSTRAP / SR_URL /
KAFKA_KEY / KAFKA_SECRET / SR_KEY / SR_SECRET / JAAS
scripts/cc-app-run.sh send orders.placed order-intake-service 'hello'   #
drives traffic with the SASL config
make cc-flink-reports-down CONFLUENT_API_KEY=... CONFLUENT_API_SECRET=...
                           # terraform destroy: deletes the environment and
everything in it
```

See the root README §4.5 "Flink SQL reports on Confluent Cloud for Apache Flink (CCAF)" for the full CCAF walkthrough, including the multi-window sustained-traffic pattern required to see tumbling-window aggregates emit.